

20233123 김남호

컴퓨터 비전 과제1 보고서

1. main 함수

```
def main():
    args = parse_args()
    img = cv2.imread(args.input)

    if img is None:
        print(f"Error: Could not read image at {args.input}", file=sys.stderr)
        return 1

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blurred = cv2.GaussianBlur(gray, (5, 5), 2)

    h, w = gray.shape[:2]
    scale = min(h, w)

    minDist = int(scale * 0.2)      # 동전 간 최소 거리 (전체 폭의 1.5%)
    minRadius = int(scale * 0.03)   # 최소 반지름 (이미지 폭의 3%)
    maxRadius = int(scale * 0.5)    # 최대 반지름 (이미지 폭의 50%)

    # 2. Hough Circle Transform 파라미터 재조정 (이중 검출 최소화)
    circles = cv2.HoughCircles(
        blurred,
        cv2.HOUGH_GRADIENT,
        dp=1.2,
        minDist=minDist,      # 이중 검출 방지를 위해 최소 거리(minDist)를 150으로 대폭 늘림
        param1=180,
        param2=35,            # 누적 임계값(param2)을 35로 올려 오검출 줄임
        minRadius=minRadius,
        maxRadius=maxRadius
    )

    # 3. 동전 분류 및 합계 계산
    coin_counts, total_sum = classify_coin(img, circles)
```

main 함수를 보면 이미지를 입력 받는다. cv.IMREAD_GRAYSCALE을 통해 그레이스케일로 이미지를 받는다. 그런 다음에 노이즈 제거를 위해 가우시안 필터링을 진행하여 원 검출의 안정성을 높였다. 커널 크기 커널 크기 (5, 5)는 인접한 5×5 픽셀 영역의 평균화를 의미하며, 세 번째 인자인 σ (sigma) 값 2는 블러 강도를 조절하는 표준편차로, 값이 커질수록 더 부드럽게 흐려진다. 그래서 (5, 5) 커널과 $\sigma=2$ 는 노이즈를 충분히 억제하면서도 동전 경계가 유지되도록 조정된 균형값이라 생각해 그렇게 설정했다.

허프 원 변환을 이용해 이미지 내 동전 영역을 검출해냈다. 이때 원 검출의 정확도를 높이기 위해 입력 영상의 크기를 기반으로 반지름 범위와 최소 간격(minDist)을 동적으로 계산했다. gray.shape[:2]를 통해 영상의 높이(h)와 폭(w)을 얻고, scale = min(h, w)으로 최소 치수를 기준 삼아 이미지 크기에 따라 비율을 맞췄다. minDist로 검출된 원 중심 간

최소 거리를 구해서 동전이 서로 겹치거나 인접해 있어도 중복해서 검출되지 않도록 조절했다. 과제에서 겹친 동전이 없기에 이렇게 중복 검출 문제를 쉽게 해결할 수 있었다. minRadius로 최소 반지름을 설정했는데 주석에서는 3%라고 했지만 실제 코드를 보면 20%다. 이렇게 하여 중복 검출 문제를 해결하려고 했으나 알다시피 동전을 멀리서 찍은 상태에서 동전 간의 간격이 좁다면 원이 검출되지 않을 수 있다는 한계가 있다. maxRadius로 최대 반지름을 구했는데 이미지 폭의 50%로 설정했다. param1과 param2는 엣지 검출 및 누적 임계값을 조정하는 주요 인자로 param1=180은 Canny 엣지 상한 값이며, param2=35는 원 검출 임계값으로 이 값을 높일수록 오검출이 줄어들 수 있도록 노력했다. 마지막으로 dp=1.2는 누적 배열의 해상도 비율로, 1보다 크면 빠르지만 부정확해지고, 1.0에 가까울수록 정밀도가 높다. 그렇지만 정확도를 높게 하기 위해 1로 했다.

다음으로 coin_counts, total_sum = classify_coin(img, circles)은 main 함수 위에 선언했는데 이를 통해 동전 크기와 색상을 이용해 얼마의 동전인지 인식하도록 했다. 이렇게 인식한 결과를 바탕으로 동전을 인식해서 얼마인지 값을 셀 수 있었다.

```
# 4. 결과 출력
print(f"500:{coin_counts[500]}")
print(f"100:{coin_counts[100]}")
print(f"50:{coin_counts[50]}")
print(f"10:{coin_counts[10]}")
print(f"{total_sum}")

return 0

if __name__ == "__main__":
    sys.exit(main())
```

2. classify_coin() 함수

coin_counts, total_sum = classify_coin(img, circles)에서 보았듯이 동전의 개수를 세고 그 총합이 얼마인지에 대한 함수 역시 구현했다. 다음은 classify_coin를 통해 이 과정을 설명하겠다.

뒷장에서 계속 >>

```

import argparse
import cv2
import sys
import numpy as np

def parse_args():
    p = argparse.ArgumentParser("CV assignment runner")
    p.add_argument("--input", required=True, type=str, help="path to input image")
    return p.parse_args()

def classify_coin(img, circles):
    coin_counts = {500: 0, 100: 0, 50: 0, 10: 0}
    total_sum = 0

    if circles is None or len(circles[0]) == 0:
        return coin_counts, total_sum

    circles = np.uint16(np.around(circles))[0]

    # 1. 색상 기반 10원 동전 분류 준비 (HSV 마스크)
    hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    # 10원 동전 (구리색/황동색) HSV 범위
    lower_color = np.array([5, 30, 30])
    upper_color = np.array([45, 255, 255])
    mask = cv2.inRange(hsv_img, lower_color, upper_color)

    # 2. 동전 분류 및 크기 그룹화 (클러스터링 대안)
    # 2-1. 검출된 원의 반지름 추출 및 정렬
    radii = [c[2] for c in circles]
    unique_radii = sorted(list(set(radii)), reverse=True) # 중복 제거 후 내림차순 정렬

    # 2-2. 반지름을 기반으로 그룹 나누기
    # 반지름 차이가 임계값 이상이면 새로운 그룹으로 간주
    TOLERANCE_RATIO = 0.05 # 반지름이 5% 이상 차이 나면 다른 동전 종류로 간주

```

먼저 coin_counts 딕셔너리를 초기화하여 각 동전(500, 100, 50, 10)의 개수를 0으로 설정한다. 만약 circles가 비어 있거나 검출된 원이 없는 경우 모든 값을 0으로 유지한 채 함수를 종료한다. 그 다음, np.around()을 통해 원 좌표와 반지름 값을 반올림하고 np.uint16()으로 정수형으로 변환하여 계산의 안정성을 높인다. 이후 입력한 이미지를 hsv 색공간으로 변환하고 10원인 구리색 동전의 hsv 범위를 설정했다. 여기에서 설정한 범위 내의 색상을 추출해서 해당 픽셀이 10원 동전 색상에 포함되는지 판별할 수 있다.

다음으로 검출한 원의 반지름을 리스트로 추출하고 중복 값을 제거한 뒤에 내림차순 정렬을 통해 unique_radii를 만들었다. 그리고 반지름이 비슷한 원들을 하나의 그룹으로 묶기 위해 반지름 차이가 평균의 5% 이상 차이가 날 경우 다른 그룹으로 간주하도록 설정했다. 이는 TOLERANCE_RATIO = 0.05로 적용했다.

뒷장에서 계속 >>

```

radius_groups = []
if unique_radii:
    current_group = [unique_radii[0]]
    for r in unique_radii[1:]:
        # 현재 그룹의 평균 반지름 대비 톨러런스 범위 내에 있는지 확인
        group_avg = np.mean(current_group)
        if r > group_avg * (1 - TOLERANCE_RATIO):
            current_group.append(r)
        else:
            radius_groups.append(current_group)
            current_group = [r]
    radius_groups.append(current_group)

# 2-3. 그룹에 동전 액면가 매핑 (가장 큰 그룹부터 500원, 100원, 50원...)
# 한국 동전의 실제 크기 순서: 500원 > 100원 > 50원 > 10원
# 매핑 순서: 500, 100, 50, 10
coin_values = [500, 100, 50, 10]

# 검출된 그룹 개수만큼만 액면가를 매핑 (없는 동전은 매핑 안됨)
# 예: 그룹이 3개면, 500, 100, 50원에 매핑
mapping = {} # {평균 반지름: 액면가}
for i, group in enumerate(radius_groups):
    if i < len(coin_values):
        mapping[np.mean(group)] = coin_values[i]
    else:
        # 4종류 이상의 동전이 검출된 경우 (오검출 가능성) 무시
        break

```

검출된 동전의 반지름 값들을 이용해 서로 비슷한 크기의 원들을 하나의 그룹으로 묶고 각 그룹을 실제 동전 액면가에 대응시키는 과정을 수행했는데 radius_groups 리스트는 이러한 그룹들을 저장하기 위한 변수다. 반지름 리스트(unique_radii)에서 첫 번째 값을 기준으로 현재 그룹(current_group)을 만들고 이후 나머지 반지름 값들에 대해 현재 그룹의 평균 반지름(group_avg)과 비교한다. 만약 새 반지름 값 r이 group_avg의 5% 이내의 차이라면 같은 그룹으로 간주하여 current_group에 추가하고 그렇지 않으면 기존 그룹을 radius_groups에 저장하고 새로운 반지름으로 새로운 그룹을 만든다. 최종적으로 모든 그룹을 리스트에 추가하면 비슷한 크기의 원들이 하나의 그룹으로 정리된다. 이후 coin_values = [500, 100, 50, 10] 리스트를 만들어, 가장 큰 반지름 그룹부터 순서대로 500원에서 100원 그리고 50원 마지막으로 10원으로 매핑하는 것이다.

뒷장에서 계속 >>

```

# 3. 모든 검출된 원을 순회하며 개수 카운트
for (x, y, r) in circles:

    # 3-1. 색상 기반 10원 동전 분류 체크 (우선순위)
    sample_radius = max(5, int(r * 0.2))
    try:
        mask_area = mask[max(0, y - sample_radius):min(img.shape[0], y + sample_radius),
                           max(0, x - sample_radius):min(img.shape[1], x + sample_radius)]
        mask_mean = np.mean(mask_area)
    except:
        mask_mean = 0

    is_10_won = False
    if mask_mean > 120: # 구리색 임계값
        # 10원 동전은 가장 작은 크기의 그룹에 속할 가능성이 높음
        if len(radius_groups) >= 4 and r < np.mean(radius_groups[3]) * 1.1:
            is_10_won = True
        elif len(radius_groups) < 4 and r < np.mean(radius_groups[-1]) * 1.1:
            # 4개 미만 그룹이지만 가장 작은 그룹과 크기가 비슷하고 색상이 10원인 경우
            is_10_won = True

    if is_10_won:
        coin_counts[10] += 1
        total_sum += 10
        continue

    # 3-2. 크기 기반 분류 (10원이 아닌 동전)
    classified = False
    min_diff = float('inf')
    assigned_value = None

```

모든 검출된 원에 대해 (x, y, r) 값을 반복하며 동전 하나씩을 판별한다. 먼저 각 원 중심 주변의 일정 영역인 $sample_radius = r \times 0.2$ 를 추출하고 해당 영역의 hsv 마스크 평균 값을 계산한다 그리고 이 값이 120 이상이면 구리색 계열로 판단하고 10원 동전 후보로 보는 것이다. 구리색이 확인될 경우 반지름이 가장 작은 그룹에 속하는지 검사하는데 만약 이 조건이 만족하면 10원으로 확정적으로 분류하는 것이고 $r < \text{평균 반지름} \times 1.1$ 만족 시 coin_counts[10]에 1을 증가시키고 total_sum에 10원을 더한다. 이렇게 색상 기반으로 분류된 동전은 이후 크기 기반 분류에서 제외한다.

뒷장에서 계속 >>


```

# 가장 가까운 그룹 평균 반지름에 매핑
for avg_r, value in mapping.items():
    if value == 10: # 10원 동전은 이미 색상으로 분류했으므로 건너뛰기
        continue

    diff = abs(r - avg_r)
    if diff < min_diff:
        min_diff = diff
        assigned_value = value

# 매핑된 값이 있고, 오차 범위 내에 있는 경우에만 카운트
if assigned_value is not None and min_diff / r < 0.05: # 5% 오차 허용
    coin_counts[assigned_value] += 1
    total_sum += assigned_value
    classified = True

return coin_counts, total_sum

```

만약 색상이 10원이 아닌 경우 반지름 차이 기반으로 분류를 진행하는데 모든 그룹 평균 반지름인 avg_r과 현재 반지름 r의 차이를 계산해서 가장 오차가 작은 그룹에 해당 동전을 매핑한다. 만약 그 오차가 반지름의 5% 이하라면 해당 그룹에 대응하는 액면가로 분류한다. 최종적으로 모든 원을 순회해서 각 액면가별 동전 개수와 총합 금액을 계산한다.

3. 한계

이번 과제는 원을 검출하고 동전의 색과 상대적 크기로 얼마인지 인식해야 했다. 물론 조건들이 예외가 없어서 쉬운 편일 수도 있었지만 생각보다 고려 해야할 조건들이 많고 여전히 밝기라던지 이미지 마다 다른 동전의 크기라던지 노이즈가 제거되지 않거나 최소 원의 크기라던지 최대 원의 크기라던지 또는 원 간의 최소한의 간격이라던지 하는 다른 많은 이유로 동전이 인식이 안될 수도 있고 또 없는 원을 찾을 수도 있었다. 하지만 이것을 머신러닝과 딥러닝을 이용해 아예 동전의 앞면과 뒷면을 인식해서 이게 얼마인지 확인할 수 있다면 좀더 정확도가 높지 않았을까 하는 아쉬움이 있었다.